

12-Tarefa: Avaliação de Escalabilidade no Supercomputador (NPAD)

1. Dados Coletados

Os testes foram executados na partição intel-128 utilizando 1 nó computacional.

Abaixo estão os tempos de execução em segundos para as diferentes estratégias de agendamento (schedule) do OpenMP variando de 1 a 16 threads, extraídos do NPAD, com tamanho de malha unificado para (NX 1024, NY 1024, STEPS 2000):

Threads	static_nocollapse (s)	static_collapse (s)	dynamic(8) (s)	guided (s)
1	17.4025	9.1055	17.3998	17.3460
2	10.5380	4.7592	9.2180	9.0758
4	5.9821	2.8034	4.9913	4.9399
8	3.4119	1.6674	2.8533	2.7671
16	1.7936	0.9031	1.4895	1.4586

2. Análise de Escalabilidade Forte (Strong Scaling)

A escalabilidade forte é avaliada mantendo o tamanho do problema **fixo** e aumentando o número de processadores. O ideal absoluto seria que, ao dobrarmos as *threads*, o tempo caísse pela metade (Speedup $S = N$).

Cálculo de Speedup ($S = T_1/T_n$) no teste para 16 Threads:

- **static:** $17.4025/1.7936 = 9.70x$
- **static_collapse:** $9.1055/0.9031 = 10.08x$
- **dynamic(8):** $17.3998/1.4895 = 11.68x$
- **guided:** $17.3460/1.4586 = 11.89x$

Discussão: As análises demonstrou excelente aderência de todas as abordagens sob a perspectiva da Escalabilidade Forte. Como a carga fixa testada (1024×1024) é densa o suficiente, observamos bons *speedups* entre $9x$ a $\approx 12x$ para 16 *threads*.

É perfeitamente observável que, na métrica do *Speedup relativo* interno de 1 vs 16 *threads*, os agendamentos **Guided e Dynamic foram mais eficientes em “ganho matemático”**, gerando quase $12x$. No entanto, o *static_collapse* permaneceu sendo o de **tempo absoluto menor na prática**, largando já da casa dos 9 segundos com apenas 1 processador.

3. Análise de Escalabilidade Fraca (Weak Scaling) teórica

A escalabilidade fraca avalia como o algoritmo se comporta se **aumentarmos a carga de trabalho na mesma proporção que aumentamos o hardware** (idealmente, o tempo deveria permanecer idêntico: 1s com 1 CPU e Malha1x, e 1s com 16 CPUs e Malha16x).

Embora o *job* executado fixou o tamanho do problema, podemos inferir o nosso grau de escalabilidade fraca pelos resultados de “overhead” já descritos. O algoritmo explícito para diferenças finitas de Navier-Stokes é perfeitamente adequado para a escalabilidade fraca caso NX e NY cresçam. Como a comunicação ocorre apenas nas bordas ou limites compartilhados (ou escrita local na cache L1/L2), caso ajustássemos o malloc dinâmico do Grid junto ao parâmetro do argv de threads, manteríamos eficiências altíssimas e tempos muito estáveis para as cláusulas `static`.

4. Identificação de Gargalos

Ao compararmos os agendamentos (`static`, `dynamic`, `guided` e `collapse`), os tempos recém-adquiridos de cargas padronizadas apontam para fatos cruciais na partição de arquitetura intel-128:

1. A Sobrecarga dos Dois Laços vs Collapse:

- A versão `static` normal perde absurdamente para a `static_collapse`. Ao invés de usar uma Thread gerindo e abrindo filas de milhares sub-laços pra varrer NY separadamente, o *collapse* achata os laços i e j. Isso cria um loop plano gigante. Evita-se completamente que as Threads saltem ou encontrem falhas prematuras de L1 e barreiras ao longo de blocos de iterações internas iterados isoladamente. Esse foi o principal ganho algorítmico frente às abordagens ingênuas.

2. Dynamic vs Guided:

- O `dynamic(8)` requer constante realocação de fatias (*chunks* de tamanho 8) a toda hora por disputa das threads ativas que ficaram velozmente desocupadas, aumentando os acessos síncronos no agendador OpenMP. O modelo `guided` provou ser melhor que ele (1.45s vs 1.48s) já que repassa partes grandonas inicialmente para esgotar as filas espaciais, poupando trabalho do “Maestro OpenMP”, e minimizando a sobrecarga no final.

Script de Submissão (job.sh)

```
#!/bin/bash
#SBATCH --job-name=navier_escala
#SBATCH --time=0-0:30
#SBATCH --partition=intel-128
#SBATCH --cpus-per-task=16
#SBATCH --hint=compute_bound

export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

# Compila os códigos
```

```

make clean
make

# Script para avaliar a escalabilidade
echo "====="
echo "Iniciando benchmarks (Escalabilidade)"
echo "Número máximo de threads do Job: $SLURM_CPUS_PER_TASK"
echo "====="

echo -e "\n=== Teste navier_serial ==="
./navier_serial

for THREADS in 1 2 4 8 16; do
    echo -e "\n====="
    echo "Testando com $THREADS Threads"
    echo "====="

    echo "--- navier_static ---"
    ./navier_static $THREADS

    echo "--- navier_static_collapse ---"
    ./navier_static_collapse $THREADS

    echo "--- navier_dynamic ---"
    ./navier_dynamic $THREADS

    echo "--- navier_guided ---"
    ./navier_guided $THREADS
done

```